Activity 10 - Plan Distributed PyTorch GPU Training

Course: Google Professional Machine Learning Engineer Training Course code: C740 Version: v2.0 | 12 September 2026 Evidence: activity artifacts

Objective

Configure torchrun-style workers, checkpoints and scaling-efficiency diagnostics.

Deliverables

- train_distributed.py
- Dockerfile
- worker_pool.yaml
- scaling_results.csv
- A completed evidence/ folder with decision notes and verification output

Prerequisites

- Python 3.11 or later and a terminal; Docker only where the activity explicitly uses it.
- For optional live Google Cloud steps: the safe training project created in Activity 01.
- Start in this activity folder so relative paths and cleanup commands remain scoped.

Safety, cost and privacy

- Use a training project that contains no production or personal data.
- Replace every placeholder such as <PROJECT_ID> and <REGION> before a cloud command.
- Do not paste API keys or service-account JSON into notebooks, scripts or screenshots.
- Prefer Cloud Shell and Application Default Credentials. Never commit .env or credentials.
- Verify the result against the acceptance checklist, then run the cleanup section.

Evidence to retain

Create an evidence/ folder locally. Save command output, job or revision IDs, metric tables, screenshots without secrets, and a short decision note. Use filenames that include the activity number and date. Evidence is part of the learning outcome, not an optional extra.

Detailed step-by-step guide

Step 1 - Read worker_pool.yaml and map chief, worker replicas, machine types, accelerator count and checkpoint URI

Read worker_pool.yaml and map chief, worker replicas, machine types, accelerator count and checkpoint URI.

Check: Record the observable result or explain the error before continuing.

Step 2 - Inspect train_distributed.py; locate process-group initialization, distributed sampler, all-reduce behavior and checkpoint save

Inspect train_distributed.py; locate process-group initialization, distributed sampler, all-reduce behavior and checkpoint save.

Check: Record the observable result or explain the error before continuing.

Step 3 - Run the local two-process smoke test without GPUs

Run the local two-process smoke test without GPUs.

```
python3 -m torch.distributed.run --standalone --nproc_per_node=2 train_distributed.py --epochs
2
```

Check: Record the observable result or explain the error before continuing.

Step 4 - Inspect scaling_results.csv and calculate speedup and scaling efficiency for 1, 2, 4 and 8 workers

Inspect scaling_results.csv and calculate speedup and scaling efficiency for 1, 2, 4 and 8 workers.

Check: Record the observable result or explain the error before continuing.

Step 5 - Explain why 8 workers may reduce efficiency even while throughput increases

Explain why 8 workers may reduce efficiency even while throughput increases.

Check: Record the observable result or explain the error before continuing.

Step 6 - Design a checkpoint interval from the observed failure rate and checkpoint write time

Design a checkpoint interval from the observed failure rate and checkpoint write time.

Check: Record the observable result or explain the error before continuing.

Step 7 - Build the custom container only if Docker is available; record the image digest

Build the custom container only if Docker is available; record the image digest.

```
docker build -t pmle-torchrun:v8 .
```

Check: Record the observable result or explain the error before continuing.

Step 8 - Propose a GPU/TPU/CPU choice using examples per dollar, framework support and quota

Propose a GPU/TPU/CPU choice using examples per dollar, framework support and quota.

Check: Record the observable result or explain the error before continuing.

Acceptance checklist

[ ] Every required artifact exists and opens without error.

[ ] Results are tied to the supplied mock data or an approved training project.

[ ] The conclusion cites metrics, IDs or command output rather than opinion alone.

[ ] No credentials, tokens, personal data or unredacted sensitive prompts are present.

[ ] Cleanup is completed or the retained resource is documented with an owner and expiry.

Troubleshooting

Symptom	Likely cause	Corrective action
403 PERMISSION_DENIED	API disabled, wrong project, or IAM role missing	Verify gcloud config get-value project, API state and the least-privilege role.
429 RESOURCE_EXHAUSTED	Quota or rate limit reached	Stop retries, inspect quota, add exponential backoff, and ask the trainer before increasing quota.
Region/location mismatch	Dataset, bucket, model or endpoint uses another location	Recreate only the training resource in one approved region; do not copy sensitive data casually.
Result differs from expected	Data, dependency or configuration version drifted	Compare file hashes, pinned versions and parameters; retain the diff as evidence.

Cleanup

Stop local processes and remove only resources created for this activity. For live cloud work, use the Console resource views and activity evidence to verify deletion. Do not delete a shared project, dataset, bucket, registry or endpoint unless the trainer identifies it as disposable.